Competitive Programming Notebook

py.Sandu

Contents

1 DP

1.1 Lis

```
// Find the length of the longest increasing
    subsequence in a given array
// Time complexity: O(n log n) where n is the length
    of the input array

int lis(vector<int>& arr) {
    vector<int> dp;
    for (int x : arr) {
        auto it = lower_bound(all(dp), x);
        if(it == dp.end()) dp.push_back(x);
        else *it = x;
    }
    return dp.size();
}
```

1.2 Countingdp

```
// Counting DP (Sum of all ways / Unbounded Knapsack)
// Time Complexity: O(N * Target)
// Space Complexity: O(Target)
// Use case: Count number of ways to form a sum

int count_ways(vector<int>& items, int target) {

    vector<int> dp(target + 1, 0);
    dp[0] = 1;

    for (int item : items) {
        for (int i = item; i <= target; i++) {
            dp[i] = (dp[i] + dp[i - item]) % MOD;
        }
    }

    return dp[target];
}
```

1.3 Knapsack

```
// General take/not take dp problem
// Time Complexity: aprox. O(n * m)
// Space Complexity: O(n * m) / O(m)
// Use cases: maximize/minimize sum

// Top-down
int knapsack_top_down(vector<int>& v, vector<int>& w,
     int x){
    int n = (int)v.size();
    vector<vector<int>> dp(n, vector<int>(x+1, -1));
    auto rec = [&](auto self, int i, int m) -> int {
        if(i >= n || m == 0) return 0;
        if(dp[i][m] != -1) return dp[i][m];
        int take = 0, not_take = self(self, i+1, m);
        if(m >= w[i]){
            take = v[i] + self(self, i+1, m - w[i]);
        }
        return dp[i][m] = max(take, not_take);
    };

    return rec(rec, 0 , x);
}

// Bottom-up
int knapsack_bottom_up(vector<int>& v, vector<int>& w
    , int x){
    int n = (int)v.size();
    vector<int> dp(x+1, 0);

    for(int i=0; i < n; i++){
        for(int m=x; m >= w[i]; m--){
            dp[m] = max(dp[m], v[i] + dp[m - w[i]]);
        }
    }

    return dp[x];
}
```

2 Math

2.1 Factorial

```
// Time Complexity: O(N) for precompute, O(1) for nCr
// Space Complexity: O(N)
// Use cases: calculating combinations, permutations,
     probabilities

const int MAXN, MOD;
vector<int> fact(MAXN), ifact(MAXN);

void precompute(){

    fact[0] = 1; ifact[0] = 1;
    for(int i=1; i<MAXN; i++){
        fact[i] = (i * fact[i-1]) % MOD;
    }

    ifact[MAXN-1] = fexp(fact[MAXN-1], MOD-2, MOD);
    for(int i=MAXN-2; i >= 1; i--){
        ifact[i] = (ifact[i+1]*(i+1)) % MOD;
    }
}

int choose(int n, int r){
    if(r < 0 or r > n) return 0;
    int num = fact[n];
    int den = (ifact[r] * ifact[n - r]) % MOD;
    return (num * den) % MOD;
}
```

2.2 Fastexponentiation

```
// Calculates (x^n) % MOD
// Time Complexity: O(log n)
// Space Complexity: O(1)

int fexp(int x, int b, int MOD){
    int res = 1;

    while(b > 0){
        if(b & 1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
    return res;
}
```

2.3 Crivo

```
// Find all prime numbers up to a given limit using
    the Sieve of Eratosthenes algorithm
// Time complexity: O(n log log n) where n is the
    limit
// This idea can also be used to construct a table
    for the smallest prime factors, O(log N) per
    query

vector<int> findPrimes(int limit) {
    vector<bool> isPrime(limit + 1, true);
    isPrime[0] = isPrime[1] = false;
```

```cpp

    for (int i = 2; i * i <= limit; i++) {
        if (isPrime[i]) {
            for (int j = i * i; j <= limit; j += i) {
                isPrime[j] = false;
            }
        }
    }

    vector<int> primes;
    for (int i = 2; i <= limit; i++) {
        if (isPrime[i]) {
            primes.push_back(i);
        }
    }
    return primes;
}

vector<int> buildSPF(int limit){
    vector<int> spf(limit+1);
    iota(spf.begin(), spf.end(), 0);
    for(int i = 2; i * i <= limit; i++) {
        if(spf[i] == i){
            for(int j = i * i; j <= limit; j += i) {
                if(spf[j] == j) spf[j] = i;
            }
        }
    }

    return spf;
}

vector<int> getFactors(vector<int>& spf, int x){
    vector<int> factors;
    while(x > 1){
        factors.push_back(spf[x]);
        x /= spf[x];
    }

    return factors;
}
```

2.4 Matrixexponentiation

```cpp
// Calculate the Nth Fibonacci number using matrix
    exponentiation.
// Time Complexity: O(log N)
// Space Complexity: O(1)

void matrixExponentiation(vector<vector<long long>>&
    matrix, int power) {
    int n = matrix.size();
    vector<vector<long long>> result(n, vector<long
    long>(n, 0));

    // Initialize result as the identity matrix
    for (int i = 0; i < n; i++) {
        result[i][i] = 1;
    }

    while (power) {
        if (power % 2 == 1) {
            matrixMultiplication(result, matrix,
    result);
        }
        matrixMultiplication(matrix, matrix, matrix);
        power /= 2;
    }

    matrix = result;
}
```

2.5 Matrixmultiplication

```cpp
// Multiply two matrices A and B, and return the
    result in matrix C.
// Time Complexity: O(n^3)
// Space Complexity: O(n^2)

vector<vector<int>> matrixMultiplication(const vector
    <vector<int>>& A, const vector<vector<int>>& B) {
    int n = A.size();
    vector<vector<int>> C(n, vector<int>(n, 0)); //
    Initialize result matrix with zeros

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
    return C;
}
```

2.6 Extendedeuclidean

```cpp
// Extended Euclidean Algorithm
// Returns (gcd(a, b), x, y) such that a*x + b*y =
    gcd(a, b)

int euclides(int a, int b, int &x, int &y){
    if(b == 0){
        x = 1, y = 0;
        return a;
    }
    int x1, y1;
    int g = euclides(b, a%b, x1, y1);
    x = y1; y = x1 - y1*(a/b);
    return g;
}

int inverso_mod(int a, int m){
    int x, y;
    int g = euclides(a, m, x, y);
    if(g != -1) return -1;
    return (x%m + m) % m;
}
```

3 String

3.1 Hash

```cpp
// String Hash template
// constructor(s) - O(|s|)
// query(l, r) - returns the hash of the range [l,r]
    from left to right - O(1)
// query_inv(l, r) from right to left - O(1)
// patrocinado por tiagodfs

mt19937 rng(time(nullptr));

struct Hash {
    const int X = rng();
    const int MOD = 1e9+7;
    int n; string s;
    vector<int> h, hi, p;
    Hash() {}
    Hash(string s): s(s), n(s.size()), h(n), hi(n), p
    (n) {
        for (int i=0;i<n;i++) p[i] = (i ? X*p[i-1]:1)
     % MOD;
        for (int i=0;i<n;i++)
```

```cpp
            h[i] = (s[i] + (i ? h[i-1]:0) * X) % MOD;
        for (int i=n-1;i>=0;i--)
            hi[i] = (s[i] + (i+1<n ? hi[i+1]:0) * X) % MOD;
    }
    int query(int l, int r) {
        int hash = (h[r] - (l ? h[l-1]*p[r-l+1]%MOD : 0));
        return hash < 0 ? hash + MOD : hash;
    }
    int query_inv(int l, int r) {
        int hash = (hi[l] - (r+1 < n ? hi[r+1]*p[r-l+1] % MOD : 0));
        return hash < 0 ? hash + MOD : hash;
    }
};
```

4 Graph

4.1 Kruskal

```cpp
// Kruskal algorithm for finding the Minimum Spanning Tree
// Time Complexity: O(E log E)
// Space complexity: O(V + E)
// Use cases: connect nodes with min/max cost, clustering, minimax path

// edges: {weight, u, v}
vector<vector<pair<int, int>>> kruskalMST(vector<array<int, 3>>& edges, int n){
    int m = (int)edges.size();
    sort(edges.begin(), edges.end());
    vector<vector<pair<int, int>>> mst(n+1);
    DSU dsu(n+1); int cost = 0;
    for(int i=0; i<m; i++){
        int w = edges[i][0], a = edges[i][1], b = edges[i][2];
        if(dsu.merge(a, b)){
            mst[a].push_back({w, b});
            mst[b].push_back({w, a});
            cost += w;
        }
    }
    return mst;
}
```

4.2 Eulereanpath

```cpp
// Find the Eulerian path in a graph using Hierholzer's algorithm
// Time complexity: O(E) where E is the number of edges in the graph
// Use cases: visit every edge once, DNA sequencing, puzzles, drawing figures

vector<int> findEulerianPath(vector<vector<int>>& adj, int start) {

    vector<int> path;
    stack<int> s;
    s.push(start);

    while (!s.empty()) {
        int i = s.top();
        if (adj[i].empty()) {
            path.push_back(i);
            s.pop();
        } else {
            int u = adj[i].back(); adj[i].pop_back();
            s.push(u);
        }
    }

    reverse(path.begin(), path.end());
    return path;
}
```

4.3 Dijkstra

```cpp
// Find shortest path from a source vertex to all other vertices in a graph with non-negative edge weights
// Time Complexity: O((V + E) log V)
// Space Complexity: O(V)
// Use cases: shortest path in non-negative weighted graphs

vector<int> dijkstra(vector<vector<pair<int, int>>>& adj, int src) {

    int n = adj.size() - 1;
    vector<int> dist(n+1, INF);
    dist[src] = 0;

    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq; // Min-heap priority queue
    pq.push({0, src});

    while (!pq.empty()) {

        auto [d, i] = pq.top();
        pq.pop();

        if(d > dist[i]) continue;

        for (auto [u, w] : adj[i]) {

            if (dist[i] + w < dist[u]) {
                dist[u] = dist[i] + w;
                pq.push({dist[u], u});
            }
        }
    }

    return dist;
}
```

4.4 Bellmanford

```cpp
// Find the shortest path from a source vertex to all other vertices in a graph with negative weight edges and detect negative weight cycles.
// Time Complexity: O(V * E)
// Space Complexity: O(V)
// Use cases: SSSP with negative edges, negative cycle detection, system of difference constraints, shortest path with at most K edges

bool bellmanFord(vector<vector<pair<int, int>>>& adj, int src) {

    int n = adj.size() - 1;
    vector<int> dist(n+1, INF);
    dist[src] = 0;

    // Relax all edges n-1 times
    for (int i = 1; i < n; i++) {
        for (int u = 1; u <= n; u++) {
            for (auto& [v, w] : adj[u]) {
                if (dist[u] != INF and dist[u] + w < dist[v]) {
```

```
                    dist[v] = dist[u] + w;
                }
            }
        }
    }

    // Check for negative weight cycles
    for (int u = 1; u <= n; u++) {
        for (auto& [v, w] : adj[u]) {
            if (dist[u] != INF and dist[u] + w < dist[v]) {
                // Graph contains negative weight cycle
                return true;
            }
        }
    }

    return false;
}
```

4.5 Floydwarshall

```
// Find the shortest paths in a weighted graph with positive or negative edge weights (but with no negative cycles).
// Time Complexity: O(V^3)
// Space Complexity: O(V^2)
// Use cases: reachability, minimax distance, global negative cycle detection(dist[i][i] < 0)

void floydWarshall(vector<vector<int>>& dist) { // dist(INF)

    int n = dist.size() - 1;
    for(int i = 1; i <= n; i++) dist[i][i] = 0;

    for (int k = 1; k <= n; k++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (dist[i][k] != INF and dist[k][j] != INF) {
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
        }
    }
}
```

4.6 Kahn

```
// Topological Sort (Kahn's Algorithm)
// Time Complexity: O(V + E)
// Space Complexity: O(V)
// Use cases: Dependency resolution, cycle detection in directed graphs, DP on DAGs

vector<int> TopologicalSort(vector<vector<int>>& adj, int n){
    vector<int> degree(n+1, 0);
    for(int i=1; i<=n; i++){
        for(auto u: adj[i]) degree[u]++;
    }

    // priority_queue in case of lexicographically smallest answer
    queue<int> q; vector<int> ans;
    for(int i=1; i<=n; i++){
        if(degree[i] == 0){
            q.push(i); ans.push_back(i);
        }
    }


    while(!q.empty()){
        int i = q.front(); q.pop();
        for(auto u: adj[i]){
            degree[u]--;
            if(degree[u] == 0){
                q.push(u); ans.push_back(u);
            }
        }
    }

    if((int)ans.size() != n) return {};
    return ans;

}
```

5 Geometry

5.1 Point

```

const double EPS = 1e-9;
// EPS = 0 -> int

struct Point {
    double x, y;

    Point() : x(0), y(0) {}
    Point(double x, double y) : x(x), y(y) {}

    Point operator+(const Point& p) const { return Point(x + p.x, y + p.y); }
    Point operator-(const Point& p) const { return Point(x - p.x, y - p.y); }
    Point operator*(double c) const { return Point(x * c, y * c); }
    Point operator/(double c) const { return Point(x / c, y / c); }

    bool operator<(const Point& p) const {
        if (abs(x - p.x) > EPS) return x < p.x;
        return y < p.y - EPS;
    }

    bool operator==(const Point& p) const {
        return abs(x - p.x) <= EPS && abs(y - p.y) <= EPS;
    }
};

// > 0: B esq. vetor OA, < 0: B dir. vetor OA, 0: Colineares
double cross_product(Point O, Point A, Point B) {
    return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
}

// distancia euclidiana
double dist(Point A, Point B) {
    return hypot(A.x - B.x, A.y - B.y);
}

// distancia quadrada
double dist2(Point A, Point B) {
    return (A.x - B.x) * (A.x - B.x) + (A.y - B.y) * (A.y - B.y);
}

// produto escalar, dot == 0 perpendiculares
double dot_product(Point O, Point A, Point B) {
```

```
    return (A.x - O.x) * (B.x - O.x) + (A.y - O.y) * (B.y - O.y);
}

int get_half(Point O, Point P) {
    if (P.y - O.y > EPS or (abs(P.y - O.y) <= EPS and P.x - O.x > EPS)) return 1;
    return 0;
}

// ordenacao por perimetro
void sort_by_angle(vector<Point>& p, Point center) {
    sort(p.begin(), p.end(), [&](const Point& A, const Point& B) {
        int halfA = get_half(center, A);
        int halfB = get_half(center, B);

        if (halfA != halfB) {
            return halfA > halfB;
        }

        double cross = cross_product(center, A, B);
        if (abs(cross) > EPS) {
            return cross > 0;
        }

        return dist2(center, A) < dist2(center, B) - EPS;
    });
}

// shoelace formula, vÃľrtices do polÃŋgono ordenados pelo perÃŋmetro.
double polygon_area(const vector<Point>& p) {
    double area = 0.0;
    int n = p.size();

    for (int i = 0; i < n; i++) {
        int next_i = (i + 1) % n;
        area += (p[i].x * p[next_i].y) - (p[next_i].x * p[i].y);
    }

    // return abs(area) -> int
    return abs(area) / 2.0;
}

// aux func
int orientation(Point O, Point A, Point B) {
    double val = cross_product(O, A, B);
    if (abs(val) <= EPS) return 0; // Colinear
    return (val > 0) ? 1 : -1;      // 1: Esquerda, -1: Direita
}

// 2. P in seg AB?
bool on_segment(Point P, Point A, Point B) {
    return P.x >= min(A.x, B.x) - EPS && P.x <= max(A.x, B.x) + EPS &&
           P.y >= min(A.y, B.y) - EPS && P.y <= max(A.y, B.y) + EPS;
}

// Ab seg cross CD?
bool segments_intersect(Point A, Point B, Point C, Point D) {
    // Calcula as 4 orientaÃğÃţes necessÃąrias
    int o1 = orientation(A, B, C);
    int o2 = orientation(A, B, D);
    int o3 = orientation(C, D, A);
    int o4 = orientation(C, D, B);

    if (o1 != o2 && o3 != o4) {
        return true;
    }

    if (o1 == 0 && on_segment(C, A, B)) return true; // C toca AB
    if (o2 == 0 && on_segment(D, A, B)) return true; // D toca AB
    if (o3 == 0 && on_segment(A, C, D)) return true; // A toca CD
    if (o4 == 0 && on_segment(B, C, D)) return true; // B toca CD

    return false; // NÃčo se cruzam
}
```

6 .template

6.1 Template

```
#include<bits/stdc++.h>

using namespace std;

#define int long long
#define vi vector<int>
#define pii pair<int, int>
#define all(x) (x).begin(), (x).end()
#define debug(x...) cout<<"["#x"]: ",[](auto...$){((cout<<$<<"; "),...);}(x),cout<<endl

void solve(){

}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int t = 1;
    //cin >> t;
    while(t--) solve();
    return 0;
}
```

7 DS

7.1 Psum2d

```
// Psum2D
// Time Complexity: Query O(1)
// Space Complexity: O(N*M)
// Use cases: static 2d range sum, fixed-size window search, max sum submatrix

int n, m;
vector<vector<int>> arr(n+1, vector<int>(m+1, 0));
vector<vector<int>> psum(n+1, vector<int>(m+1, 0));

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        cin >> arr[i][j];
        psum[i][j] =  arr[i][j]
                    + psum[i][j-1]
                    + psum[i-1][j]
                    - psum[i-1][j-1];
    }
}

// canto superior esquerdo (x1, y1) canto inferior direito (x2, y2)
int x1, y1, x2, y2;
```

```cpp
int res =   psum[x2][y2]
          - psum[x1 - 1][y2]
          - psum[x2][y1 - 1]
          + psum[x1 - 1][y1 - 1];
```

7.2 Sparsetable

```cpp
// Sparse Table
// Time Complexity: O(n * log n) build, O(1) querie
// Space complexity: O(n * log n)
// Idempotent operation f(a, a) = a (min, max, gcd, and, or)
// Use cases: idempotent functions range queries

struct SparseTable{

    int n, p;
    vector<vector<int>> st;

    SparseTable(vector<int> &arr){
        n = (int) arr.size();
        p = __lg(n);
        st.assign(p+1, vector<int>(n, 0));

        for(int j=0; j<n; j++) st[0][j] = arr[j];
        for(int i = 1; i <= p; i++){
            int k = (1ll << (i-1));
            int lim = n - 2*k + 1;
            for(int j=0; j<lim; j++){
                st[i][j] = op(st[i-1][j], st[i-1][j+k]);
            }
        }
    }

    int query(int l, int r){
        int len = r  - l + 1;
        int p_local = __lg(len), k = (1ll << p_local);
        return op(st[p_local][l], st[p_local][r - k + 1]);
    }

};
```

7.3 Segtree

```cpp
template <typename T>

struct SegTree{
    int n;
    vector<T> tree;
    const T NEUTRO = 1e18;

    T merge(T a, T b) {return min(a, b);}
    SegTree(int n){
        this->n = n;
        tree.assign(2*n, NEUTRO);
    }

    void build(vector<T>& v){
        for(int i=0; i<n; i++) tree[n+i] = v[i];
        for(int i=n-1; i>0; i--) tree[i] = merge(tree[2*i], tree[2*i + 1]);
    }

    void update(int pos, T val){
        pos += n;
        tree[pos] = val;
        for(pos /= 2; pos > 0; pos /= 2) tree[pos] = merge(tree[pos*2], tree[pos*2 + 1]);
    }

    T query(int l, int r){
        T res_l = NEUTRO, res_r = NEUTRO;
        for(l += n, r += n; l <= r; l /= 2, r /= 2){
            if(l%2 != 0) res_l = merge(res_l, tree[l++]);
            if(r%2 == 0) res_r = merge(tree[r--], res_r);
        }
        return merge(res_l, res_r);
    }

};
```

7.4 Bit2d

```cpp
// Binary Indexed Tree(BIT) 2D
// Time complexity: query and update log N * log M
// Space complexity: O(N*M)
// Use cases: dynamic 2D prefix sums, counting active points in a dynamic rectangle

struct BIT2D {

    int n, m; vector<vector<int>> bit;
    BIT2D(int n, int m) : n(n), m(m) {
        bit.assign(n+1, vector<int>(m+1, 0));
    }

    void update(int x, int y, int val) {
        for (int i = x; i <= n; i += i & -i) {
            for (int j = y; j <= m; j += j & -j) {
                bit[i][j] += val;
            }
        }
    }

    int query(int x, int y) {
        int sum = 0;
        for (int i = x; i > 0; i -= i & -i) {
            for (int j = y; j > 0; j -= j & -j) {
                sum += bit[i][j];
            }
        }
        return sum;
    }

    int queryRange(int x1, int y1, int x2, int y2) {
        return query(x2, y2)
             - query(x1 - 1, y2)
             - query(x2, y1 - 1)
             + query(x1 - 1, y1 - 1);
    }

};
```

7.5 Dsu

```cpp
// Disjoint Set Union(DSU)
// Time Complexity: aprox. O(1)
// Space Complexity: O(n)
// Use cases: connected components, cycle detection, offline edge deletion

struct DSU{

    vector<int> parent, size;

    DSU(int n){
        parent.assign(n+1, 0);
        size.assign(n+1, 1);
        iota(parent.begin(), parent.end(), 0);
```

```
    }

    int find(int v){
        if(v == parent[v]) return v;
        return parent[v] = find(parent[v]);
    }

    bool same(int a, int b){
        return find(a) == find(b);
    }

    bool merge(int a, int b){
        a = find(a);
        b = find(b);
        if(a != b){
            if(size[a] < size[b]) swap(a, b);
            parent[b] = a;
            size[a] += size[b];
            return true;
        }
        return false;
    }

};
```

7.6 Bit

```
// Binary Indexed Tree(BIT)
// Time complexity: query and update log N
// Space complexity: O(N)
// Use cases: dynamic psum, inversion counting, dynamic multiset

struct BIT{

    int n; vector<int> bit;
    BIT(int n) : n(n){
        bit.assign(n+1, 0);
    }

    void update(int i, int x){
        while(i <= n){
            bit[i] += x;
            i += i & -i;
        }
    }

    int query(int i){
        int sum = 0;
        while(i > 0){
            sum += bit[i];
            i -= i & -i;
        }
        return sum;
    }

    int queryRange(int l, int r){
        return query(r) - query(l-1);
    }

    // retorna o Ãŋndice do k-Ãľsimo elemento da bit
    int find_kth(int k){

        int idx = 0;
        int max_bit = __lg(n);

        for(int i = max_bit; i>= 0; i--){
            int next_idx = idx + (1ll << i);
            if(next_idx <= n and bit[next_idx] < k){
                idx = next_idx;
                k -= bit[next_idx];
            }
        }

        return idx+1;
    }

};
```

7.7 Seglazy

```
template <typename T>

struct SegLazy{
    int n;
    vector<T> tree, lazy;
    const T NEUTRO = 0, LAZY_NEUTRO = 0;

    T merge(T a, T b) {return a + b;}

    SegLazy(int n){
        this->n = n;
        tree.assign(4*n, NEUTRO);
        lazy.assign(4*n, NEUTRO);
    }

    void push(int node, int l, int r){
        if(lazy[node] == LAZY_NEUTRO) return;
        tree[node] += lazy[node] * (r - l + 1);
        if(l != r){
            lazy[2*node] += lazy[node];
            lazy[2*node + 1] += lazy[node];
        }
        lazy[node] = LAZY_NEUTRO;
    }

    void build(int node, int l, int r, vector<T> &v){
        if(l == r){
            tree[node] = v[l]; return;
        }
        int mid = (l+r)/2;
        build(2*node, l, mid, v);
        build(2*node + 1, mid+1, r, v);
        tree[node] = merge(tree[2*node], tree[2*node + 1]);
    }

    void update(int node, int l, int r, int ql, int qr, T val){
        push(node, l, r);
        if(ql > r or qr < l) return;
        if(ql <= l and r <= qr){
            lazy[node] += val;
            push(node, l, r);
            return;
        }
        int mid = (l+r)/2;
        update(2*node, l, mid, ql, qr, val);
        update(2*node + 1, mid+1, r, ql, qr, val);
        tree[node] = merge(tree[2*node], tree[2*node + 1]);
    }

    T query(int node, int l, int r, int ql, int qr){
        push(node, l, r);
        if(ql > r or qr < l) return NEUTRO;
        if(ql <= l and r <= qr) return tree[node];
        int mid = (l+r)/2;
        return merge(query(2*node, l, mid, ql, qr), query(2*node + 1, mid + 1, r, ql, qr));
    }

    void build(const vector<T>& v) { build(1, 0, n - 1, v); }
    void update(int ql, int qr, T val) { update(1, 0, n - 1, ql, qr, val); }
```

```
    T query(int ql, int qr) { return query(1, 0, n - 1, ql, qr); }


};
```

7.8 Monotonicstack

```
// Monotonic Stack, next greater number in a circular array
// Time Complexity: O(N)
// Space Complexity: O(N)
// Use cases: ring problems, finding next larger obstacle in a loop

for (int i = 0; i < 2 * n; i++) {
    int num = a[i % n];
    while (!st.empty() and a[st.top()] < num) { // remove os menores
        res[st.top()] = num; st.pop();
    }

    if (i < n) st.push(i);
}

// Monotonic Stack, next smaller element in a linear array
// Use cases: largest rectangle in histogram, subarray contribution

for (int i = 0; i < n; i++) {
    while (!st.empty() and a[st.top()] > a[i]) { // remove os maiores
        res[st.top()] = a[i];
        st.pop();
    }
    st.push(i);
}
```